現代與後量子密碼學導論

Introduction to Modern and Post-quantum Cryptography

Fall 2026

Part 1: Provable Security

1.5: Chosen-Plaintext & Chosen-Ciphertext Attacks

Zi-Yuan Liu

CC BY NC SA

The original slides were prepared by Nadim Kobeissi.

Section 1

Chosen-Plaintext Attacks

CPA Security

Security Against Chosen-Plaintext Attacks (CPA Security)

Let Σ be an encryption scheme, and let $\Sigma.\mathcal{C}(\ell)$ denote the set of possible ciphertexts for plaintexts of length ℓ. If Σ supports only plaintexts of a single length, we can simply write $\Sigma.\mathcal{C}$ to denote the entire set of ciphertexts.[a]

Σ has security against **chosen-plaintext attacks** if the following two libraries are indistinguishable:

[a] i.e., "just forget about the length".

$\mathcal{L}^{\Sigma}_{\text{cpa-real}}$

$K \twoheadleftarrow \Sigma.\mathcal{K}$

$\underline{\text{CPA.ENC}(M)}$:

$C \coloneqq \Sigma.\text{Enc}(K, M)$

return C

$\approx$

$\mathcal{L}^{\Sigma}_{\text{cpa-rand}}$

$\underline{\text{CPA.ENC}(M)}$:

$C \twoheadleftarrow \Sigma.\mathcal{C}(|M|)$

return C

Why CPA security matters

- CPA security means: *"Even if I let you encrypt any message you want, you can't obtain any distinguisher regarding my scheme."*
- **Why this matters in real life:**
  - Attackers can often trick systems into encrypting data they choose.
  - Without CPA security, seeing these encryptions could reveal your secrets.
  - Example: If identical bank transactions always encrypt to identical ciphertexts, attackers could identify your purchases.
- The only thing that's allowed to leak is the length of messages/ciphertexts.
- Most modern encryption is designed to have this important property.

Message length: not important, not unimportant

- Even with CPA security, the **length** of messages is still leaked.
- This seemingly minor leak can reveal surprising information:
  - **Encrypted VoIP calls**: Length patterns can reveal which language is spoken.
  - **Encrypted web traffic**: Sizes of requests/responses can identify websites.
  - **Encrypted messages**: Length patterns can reveal the type of content (document, image, etc.).
  - **Encrypted commands**: Length often reveals which command was issued.
- Mitigation typically involves padding messages to fixed lengths or standard increments.
- This is why many secure protocols use fixed-size packets or add random padding.

CPA security is why we avoid deterministic encryption

- Deterministic encryption will always fail CPA security!
- If the same message always encrypts to the same ciphertext:
  - Attackers can recognize repeated messages.
  - Can build a “dictionary” of known plaintexts.
  - etc.
- ECB mode is a classic example of insecure deterministic encryption.

$\mathcal{A}$

$M \twoheadleftarrow \mathcal{M}$

$C_1 := \text{CPA.ENC}(M)$

$C_2 := \text{CPA.ENC}(M)$

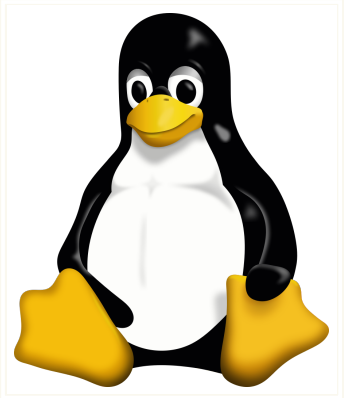
return $C_1 \overset{?}{=} C_2$

AES is a block cipher

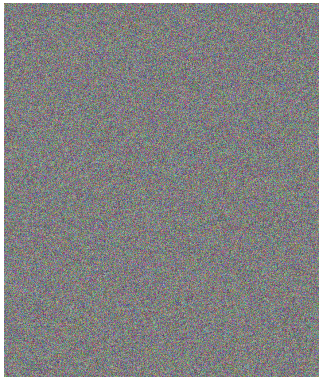
Reminder

- AES takes a 16-byte input, produces a 16-byte output.
- Key can be 16, 24, or 32 bytes.
- OK, so what if we want to encrypt more than 16 bytes?
- **Proposal**: split the plaintext into 16-byte chunks, encrypt each of them with the same key.

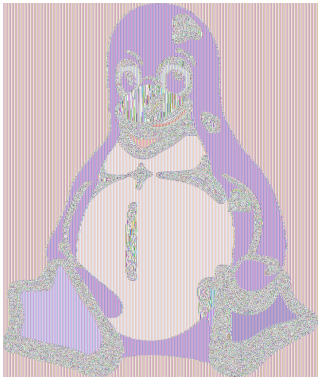
Block cipher examples



What we start with



What we want



What we actually get

AES by itself is deterministic

- AES as a block cipher is inherently **deterministic**:
  - $\mathsf{AES}(K, M)$ will always produce the same C.
  - This makes raw AES fail CPA security.
- What makes AES secure in practice?
  - **Block cipher modes** (CBC, CTR, GCM) add randomness/state using "initialization vectors".
    - IV is a fancy name for "random bytes used once".[a]
  - IVs ensure the same plaintext encrypts differently each time.
- Without a proper mode, AES is like ECB mode: predictable patterns.

$\mathcal{A}_{\mathsf{AES}}$

$K \twoheadleftarrow \{0,1\}^{\lambda}$

$M \twoheadleftarrow \mathcal{M}$

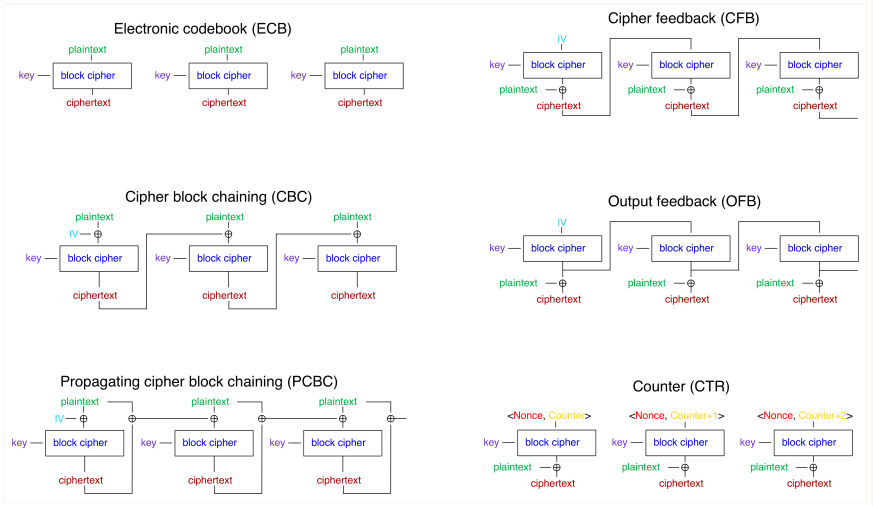
$C_1 := \mathsf{AES}(K, M)$

$C_2 := \mathsf{AES}(K, M)$

return $C_1 \overset{?}{=} C_2$

[a] Also referred to as "nonces" for "number used once". Note: in CBC mode the IV must be *unpredictable*, while in CTR mode the nonce must be *unique* but need not be random.

Block cipher modes of operation



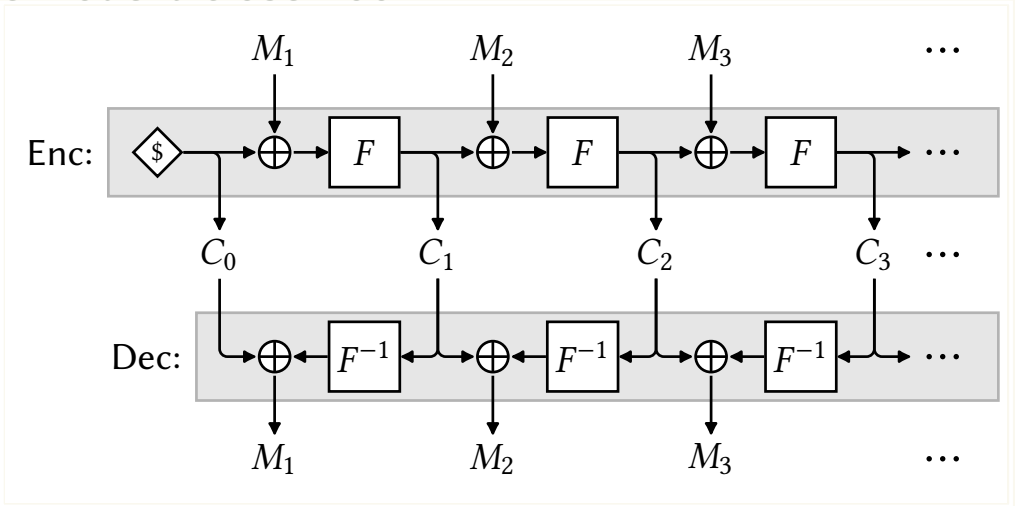
Source: Wikipedia

CBC mode: a closer look

- CBC (Cipher Block Chaining) uses the previous ciphertext block to randomize the current block.
- Requires a random initialization vector (IV).
- Each block's encryption depends on all previous blocks.
- Changes to one block affect all subsequent ciphertext blocks.
- Sequential encryption (**can't parallelize**).

$\underline{\text{ENC}(K, M_1 \| \cdots \| M_\ell):}$
$C_0 \twoheadleftarrow \{0,1\}^\lambda$
for $i = 1$ to ℓ :
  $C_i \coloneqq F(K, C_{i-1} \oplus M_i)$
return $C_0 \| C_1 \| \cdots \| C_\ell$

CBC mode: a closer look



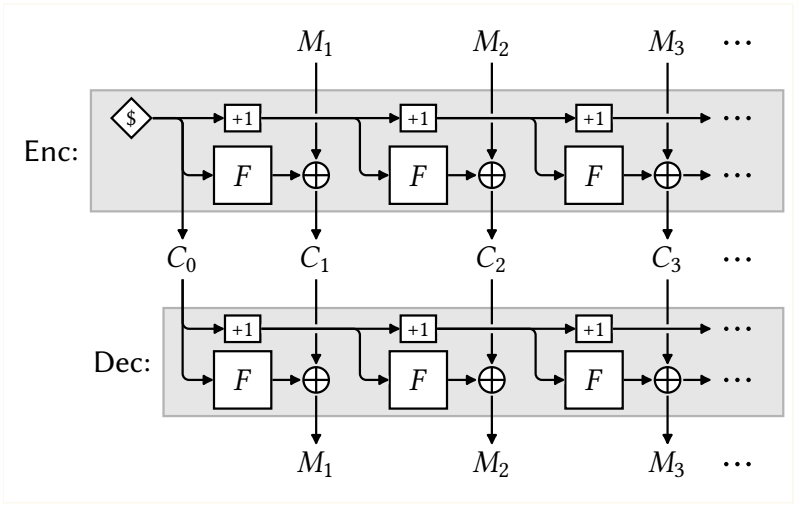
Source: The Joy of Cryptography

CTR mode: a closer look

- CTR (Counter) mode turns a block cipher into a stream cipher.
- Uses a nonce (C_0) plus a counter to create unique inputs to F.
- Each block's encryption is independent of other blocks.
- **Highly parallelizable** (unlike CBC).
- The nonce must never be reused with the same key.
- Widely used in modern protocols (TLS, SSH, etc.).

$\underline{\text{ENC}(K, M_1 \| \cdots \| M_\ell):}$
$C_0 \twoheadleftarrow \{0,1\}^\lambda$
for $i = 1$ to ℓ :
$\quad C_i := F(K, C_0 + i - 1) \oplus M_i$
return $C_0 \| C_1 \| \cdots \| C_\ell$

CTR mode: a closer look



Source: The Joy of Cryptography

Section 2

Chosen-Ciphertext Attacks

Introducing chosen-ciphertext attacks

- Imagine this scenario at your company:
  - You discover a bug: the software crashes when decrypting ciphertexts that yield plaintexts with null characters.
  - You think: “We never encrypt messages with null bytes, so no problem!”
- An attacker can:
  - Send specially crafted ciphertexts to your system.
  - Observe which ones crash your software (contain null bytes) and which don’t.
  - Use this information to completely break your encryption scheme!

Introducing chosen-ciphertext attacks

- This is a **chosen-ciphertext attack** (CCA):
  - The attacker can submit arbitrary ciphertexts for decryption.
  - The system leaks information about the decryption results (the crash).
  - Even small information leaks can completely compromise security.
  - All encryption schemes we've seen so far are vulnerable to this!

CPA vs. CCA security definitions

CPA Security (informal)

"Even if I let you encrypt any message you want, you can't obtain any distinguisher regarding my scheme."

CCA Security (informal)

"Even if I let you encrypt any message you want AND decrypt any ciphertext you want, you can't obtain any distinguisher regarding my scheme."

CPA vs. CCA security definitions

To be more precise...

CPA Security (more precise)

"Even if I let you choose any messages to encrypt, you can't tell which of two chosen messages my challenge ciphertext encrypts, except with probability close to $\frac{1}{2}$."

CCA Security (more precise)

"Even if I let you choose any messages to encrypt and decrypt (except the challenge ciphertext itself), you can't tell which of two chosen messages my challenge ciphertext encrypts, except with probability close to $\frac{1}{2}$."

Format-oracle attacks & malleability

- Scenario: a victim decrypting CTR-mode ciphertexts reveals whether the result contains a null character.
- Information can leak through crashes, error messages, or behavior differences.
- A single yes/no bit of information can be enough to break encryption!
- Malleability: many encryption schemes allow attackers to make predictable changes to plaintexts by modifying ciphertexts.
- CTR mode is especially vulnerable because XOR allows targeted bit-flipping.

$\underline{\text{NULLORACLE}(C)}$:
$M \coloneqq \texttt{CTR.Dec}(K, C)$
if M contains a `00` character:
    return `true`
else return `false`

Why is it called $\textsc{nulloracle}(C)$?

- In mythology and colloquial language:
  - An oracle was a mystic who answered questions on behalf of the gods.
  - Often gave enigmatic answers requiring interpretation.



The Oracle by Camillo Miola, 1880[a]

[a] Absolutely beautiful painting, well worth viewing in high resolution: https://upload.wikimedia.org/wikipedia/commons/9/94/Camillo_Miola_%28Biacca%29_-_The_Oracle_-_72.PA.32_-_J._Paul_Getty_Museum.jpg

Why is it called $\textsc{nulloracle}(C)$?

- In cryptography and computer science:
  - An oracle is an algorithm that solves a specific problem.
  - You can query it but can't see its internal workings.
  - Oracles are often used as theoretical tools in security proofs.
- Our $\textsc{nulloracle}(C)$ reveals only:
  - Whether a decrypted ciphertext contains null (`00`) characters.
  - Just a single bit of information (yes/no).

Null-oracle attack

- An adversary who has access to $\textsc{nulloracle}(C)$ can efficiently compute $\texttt{Dec}(K, C)$ for any C!
- Yes, it really is enough.

$\textsc{nulloracle}(C)$:

$M := \texttt{CTR.Dec}(K, C)$

if M contains a `00` character:

    return `true`

else return `false`

CTR mode is malleable

Defining malleability

Malleability

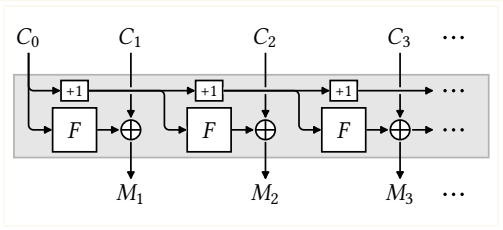
An encryption scheme is **malleable** if, given an encryption C of an unknown plaintext M, it is possible to create a new ciphertext $C' \neq C$ where $M' = \mathtt{Dec}(K, C')$ is somehow related to M, so that M' reveals some information about M.

- In CTR mode: $C_i = F(K, C_0 + i - 1) \oplus M_i$
- Flipping a bit in C_i flips exactly the same bit in M_i.
- Attackers can make targeted modifications without knowing the key.
- Example: change “transfer $100” to “transfer $900” by modifying just one byte.

CTR mode is malleable

Defining malleability

- In CTR mode:
  $C_i = F(K, C_0 + i - 1) \oplus M_i$
- Flipping a bit in C_i flips exactly the same bit in M_i.
- Attackers can make targeted modifications without knowing the key.
- Example: change “transfer $100” to “transfer $900” by modifying just one byte.

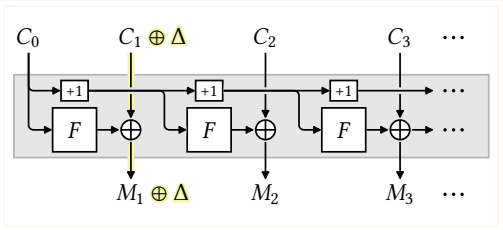


CTR mode decryption.
Source: The Joy of Cryptography

CTR mode is malleable

Defining malleability

- In CTR mode:
  $C_i = F(K, C_0 + i - 1) \oplus M_i$
- Flipping a bit in C_i flips exactly the same bit in M_i.
- Attackers can make targeted modifications without knowing the key.
- Example: change “transfer $100” to “transfer $900” by modifying just one byte.



CTR mode decryption.
Source: The Joy of Cryptography

CTR mode is malleable

The attack

- Suppose adversary captures ciphertext C and wants to find plaintext $M = \texttt{Dec}(K, C)$.
- Let's focus on discovering just the last byte of M.
- If the last byte of M is b, then $M = M' \| b$.
- What if we modify the ciphertext by XORing the last byte with b?
- This would turn the last byte of the plaintext into all zeros (null byte)!

$C \oplus (\ldots \texttt{00} \| b)$ decrypts to:

$$
\begin{aligned}
& M \oplus (\ldots \texttt{00} \| b) \\
&= (M' \| b) \oplus (\ldots \texttt{00} \| b) \\
&= M' \| (b \oplus b) \\
&= M' \| \texttt{00}
\end{aligned}
$$

CTR mode is malleable

Finding one byte

- Problem: We don't know b in advance!
- Solution: Try all 255 possible values for b.
- For each guess g, we:
  - Create $C' = C \oplus (\ldots \texttt{00} \| g)$.
  - Send C' to the NULLORACLE().
  - If the oracle returns `true`, then $g = b$!
- When we find the right value, we've discovered the last byte of M.

Try these ciphertexts:

$$C \oplus (\ldots \texttt{00} \| \texttt{01})$$
$$C \oplus (\ldots \texttt{00} \| \texttt{02})$$
$$C \oplus (\ldots \texttt{00} \| \texttt{03})$$
$$\vdots$$
$$C \oplus (\ldots \texttt{00} \| \texttt{fe})$$
$$C \oplus (\ldots \texttt{00} \| \texttt{ff})$$

CTR mode is malleable

Finding all bytes

- We can use the same approach for any byte in the plaintext.
- To find the i-th byte:
  - Create a string of all zeros.
  - Set only the i-th byte to our guess g.
  - XOR this with the ciphertext.
  - Query the oracle.
- By repeating for all bytes, we can recover the entire plaintext!

$\mathcal{A}_{\text{null-oracle}}$

$n \coloneqq |C_1\| \cdots \|C_\ell|$
$M \coloneqq$ ""

for $i = 1$ to n:
  for each $b \in \{$`01`, … , `ff`$\}$:
    $\Delta \coloneqq$ (`00`)^n
    $\Delta[i] \coloneqq b$
    if $\text{NULLORACLE}(C \oplus (0^\lambda \| \Delta))$:
      $M \coloneqq M \| b$
      next i
return M

CTR mode is malleable

The power of chosen-ciphertext attacks

- With at most 255 queries per byte, we can completely decrypt any ciphertext!
- For context: decrypting a 1KB file would take about 250,000 queries.
- This is extremely practical for an attacker.
- All from a single bit of information about the plaintext (contains null or not).
- This attack works because:
  - CTR mode is malleable (we can make predictable changes to the plaintext).
  - The system leaks a tiny bit of information about decrypted plaintexts.
  - Together, these flaws completely break the encryption.

Is the null-oracle attack just brute-force?

- Yes and no!
- It's brute-force on each byte independently:
  - To recover an n-byte plaintext: at most $255n$ oracle queries.
  - True brute-force on the entire plaintext: 256^n (exponentially worse!)
- For a 16-byte message:
  - Null-oracle attack: $\approx 4{,}080$ queries (16×255)
  - True brute-force: $\approx 10^{38}$ queries (256^{16})
- This attack is exponentially more efficient than traditional brute-force.

Can we just rate-limit the number of queries?

- Rate-limiting might help, but:
  - It only increases attack time; it doesn't prevent the attack.
  - Attackers can be patient or use multiple accounts.
  - Legitimate users suffer from the rate-limiting.
- Better approach: cryptographic solutions!
  - Fix the fundamental vulnerability, not just limit its exploitation.
  - Create systems that are mathematically proven to resist chosen-ciphertext attacks.



Is fixing the null-byte bug enough?

- The null-character bug is just one example of a broader class of attacks.
- Format-oracle attacks can exist without implementation bugs!
- They only need:
  - A system that accepts untrusted ciphertexts
  - Decrypts them
  - Behaves differently based on the decryption result
  - In a way the attacker can observe
- This behavior can come from:
  - Accidental bugs (null-byte crashes)
  - Intentional features (error messages, timing differences)
  - Normal application logic (a web app behaving differently based on decrypted data)

Real-world format-oracle attacks

Padding oracle

- CBC mode requires padding to handle plaintexts that aren't block-aligned.
- Many implementations crash when encountering invalid padding.
- This exposes an oracle that tells attackers: "Does $\mathsf{Dec}(K, C)$ have valid padding?"
- Attackers can systematically exploit this to decrypt arbitrary ciphertexts.
- This has led to major vulnerabilities in SSH and SSL/TLS protocols.
- Example: The POODLE attack against SSL 3.0 affected millions of websites.

Real-world format-oracle attacks

Timing side-channel

- The victim interprets the plaintext as a number n and performs n operations.
- Attackers can measure how long the system takes to respond.
- The response time reveals approximate numerical values inside $\mathrm{Dec}(K, C)$.
- Extremely subtle - even microsecond differences can leak information.
- Successfully used to break older SSH and SSL/TLS implementations.
- Example: The Lucky Thirteen attack against TLS revealed message contents through timing differences.

Real-world format-oracle attacks

Apple iMessage

- Older Apple iMessage versions used gzip compression.
- The system responded differently when a ciphertext decrypted to:
  - A valid gzip file (processed normally)
  - An invalid gzip file (error reported)
- This created an oracle revealing: "Is $\mathtt{Dec}(K, C)$ a valid gzip file?"
- Attackers who understood the gzip format could exploit this to:
  - Silently recover private messages
  - Bypass encryption entirely

Real-world format-oracle attacks

XML format oracles

- Many systems decrypt data expecting valid XML format.
- If decrypted data isn't valid XML, the system returns an error.
- This exposes an oracle: "Is $\mathsf{Dec}(K, C)$ valid XML?"
- XML has complex syntax rules that attackers can exploit.
- Can allow complete decryption of arbitrary ciphertexts.
- Similar attacks exist for other formats (JSON, HTML, etc.).

I thought CTR mode was secure?

CPA vs. CCA security

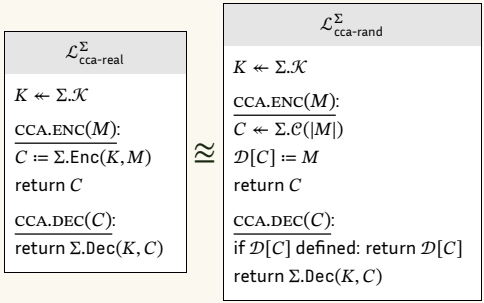
- CTR mode *is* secure against chosen-plaintext attacks (CPA-secure).
  - It uses randomness to ensure identical messages encrypt differently each time.
  - The adversary cannot distinguish its ciphertexts from random, even for chosen plaintexts.
- But CPA security isn't enough in many real-world scenarios!
  - CPA security only considers attackers who can request encryptions.
  - It doesn't protect against attackers who can submit chosen ciphertexts.
- In the null-oracle attack:
  - The victim decrypts ciphertexts chosen by the adversary.
  - Even leaking one bit about the plaintext (contains nulls or not) is fatal.
  - CPA security doesn't model or prevent this type of attack.
- This is why we need stronger security notions (CCA security).

CCA security: now you also get a decryption oracle

CCA Security

Σ is **CCA-secure** if these two libraries are indistinguishable:

- The adversary now gets a *decryption* oracle too — not just encryption.
- The ideal $\mathcal{L}^{\Sigma}_{\text{cca-rand}}$ decrypts only what it created (via dictionary $\mathcal{D}$); tampered ciphertexts fall through to the real Dec, so any information the scheme leaks on them becomes visible to the adversary.



Malleability breaks CCA — the one you've already met

- Take our CPA-secure PRF scheme. It is **not** CCA-secure.
- Recall CTR's malleability: flipping a bit in the ciphertext flips the *same* bit in the plaintext. This scheme has it too — S is just M XOR'd with a keystream.
- **The recipe** (any malleable scheme): find a ciphertext tweak that makes a *predictable* change to the plaintext, then let the decryption oracle confirm it.
- Here: XOR any Δ into S, and the decryption comes back as $M \oplus \Delta$.

$\underline{\text{ENC}(K, M)}$:
$R \twoheadleftarrow \{0,1\}^\lambda$
$S := F(K, R) \oplus M$
return $R \| S$

$\underline{\text{DEC}(K, R\|S)}$:
$M := F(K, R) \oplus S$
return M

One bit of malleability already breaks CCA

- Encrypt M, maul the ciphertext by XORing Δ into S, then ask for a decryption:

$\mathcal{A}$
$M \twoheadleftarrow \{0,1\}^\lambda$
$\Delta \coloneqq$ arbitrary nonzero string
$R \| S \coloneqq \text{CCA.ENC}(M)$
$M' \coloneqq \text{CCA.DEC}(R \| (S \oplus \Delta))$
return $M' \overset{?}{=} M \oplus \Delta$

- **Real library:** $M' = M \oplus \Delta$ every time, so it returns `true` with probability 1.
- **Ideal ($\mathcal{L}_{\text{cca-rand}}$):** the mauled ciphertext isn't in $\mathcal{D}$, so M' is unrelated to M — `true` with probability $\approx \frac{1}{2^\lambda}$.
- One predictable relationship is all it takes. **Not CCA-secure.**

Forging ciphertexts you never sent

The Frankenstein trick, part 1

- A tougher scheme:
  $S = F(K, R) \oplus F(K, M)$, and decryption *inverts* the PRF — so F must be a PRP like AES.
- No single-bit maul works here — so we get sneakier.
- **The recipe**: request two encryptions (use the *same* M), then mix and match their halves into "Frankenstein" ciphertexts.

$\mathcal{L}_{\text{cca-real}}$

$K \twoheadleftarrow \{0,1\}^\lambda$

$\underline{\text{CCA.ENC}(M)}$:
$R \twoheadleftarrow \{0,1\}^\lambda$
$S := F(K, R) \oplus F(K, M)$
return $R \| S$

$\underline{\text{CCA.DEC}(R \| S)}$:
$M := F^{-1}(K, F(K, R) \oplus S)$
return M

Forging ciphertexts you never sent

The Frankenstein trick, part 2

- Encrypt the same M twice: $R_1 \| S_1$ and $R_2 \| S_2$.
- Swap the halves and decrypt $R_1 \| S_2$ and $R_2 \| S_1$. Both unwrap to the *same* value:

$$F^{-1}\big(K,\ F(K,R_1) \oplus F(K,R_2) \oplus F(K,M)\big).$$

- Equal outputs can't happen in the ideal world (only w.p. $\approx \frac{1}{2^\lambda}$). Distinguished — and forged.

$\mathcal{A}$

$M \twoheadleftarrow \{0,1\}^\lambda$
$R_1 \| S_1 := \text{CCA.ENC}(M)$
$R_2 \| S_2 := \text{CCA.ENC}(M)$
$M_1^* := \text{CCA.DEC}(R_1 \| S_2)$
$M_2^* := \text{CCA.DEC}(R_2 \| S_1)$
return $M_1^* \stackrel{?}{=} M_2^*$

Why this matters

- Reading plaintext is bad. **Minting valid ciphertexts is worse.**
- Suppose a bank encrypts transactions with the Frankenstein scheme:
  - The attacker intercepts a few encrypted transactions.
  - Mixing their halves forges *valid* ciphertexts for transactions that never happened.
  - The system decrypts them without error and accepts them.
- Both attacks ride on the same flaw: **malleability**.

The fix

Reject any ciphertext we didn't create ourselves.

That is exactly what **authenticated encryption** gives us, and it's where we're headed next.

Section 2

Chosen-Ciphertext Attacks

Subsection 2.1

Message Authentication Codes

Symmetric primitive example: hash functions

Reminder

Hash Function Properties

- Takes input of **any size**
- Produces output of **fixed size**
- Is **deterministic** (same input → same output)
- Even a **tiny change** in input creates completely different output
- Is **efficient** to compute

SHA256(hello) =
2cf24dba5fb0a30e26e83b2ac5
b9e29e1b161e5c1fa7425e7304
3362938b9824

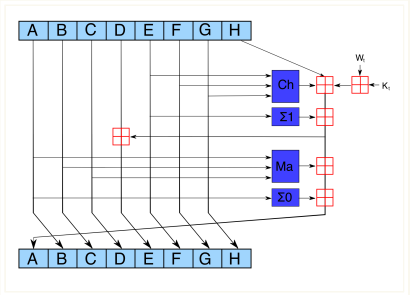
SHA256(hullo) =
7835066a1457504217688c8f5d
06909c6591e0ca78c254ccf174
50d0d999cab0

Note: One character change → completely different hash!

Expected properties of a hash function

Reminder

- **Collision resistance**: computationally infeasible to find two different inputs producing the same hash.
- **Preimage resistance**: given the output of a hash function, it is computationally infeasible to reconstruct the original input.
- **Second preimage resistance**: given an input and an output, it's computationally infeasible to find another different input producing the same output.



SHA-2 compression function. Source: Wikipedia

Hash functions: what are they good for?

Reminder

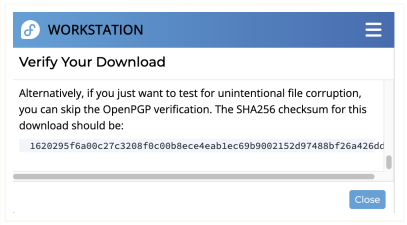
- **Data integrity verification**: Hash a file. Later hash it again and compare hashes to check if the file has changed, suffered storage degradation, etc.
- **Proof of work**: Server asks client to hash something a lot of times before they can access some resource. Useful for anti-spam, Bitcoin mining, etc.

Data integrity: verifying a Linux ISO download

- Linux distributions publish a hash (usually SHA-256) for each ISO image they release.
- After downloading, you hash your own copy and compare it to the published value:

$$\texttt{SHA256}(\texttt{your.iso}) \stackrel{?}{=} \texttt{1620295f}\ldots$$

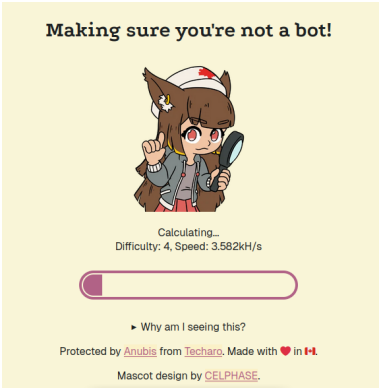
- A match means the file arrived **intact**: no corruption from a flaky network or a failing disk.
- This only catches *accidental* corruption: a malicious mirror could swap both the ISO *and* the hash, which is why real verification also uses a signature (OpenPGP).



Fedora's download page showing the expected SHA-256 checksum.

Proof of work: Anubis

- Anubis shields web servers from aggressive bots (e.g. AI scrapers) by making each visitor's browser solve a hash puzzle.
- **The challenge**: given a value C, find a nonce N such that SHA256($C \| N$) begins with a set number of leading zero bits (the *difficulty*).
- Finding N takes brute force over many guesses, but checking a submitted N is one hash: **hard to solve, easy to verify**.
- A human barely notices the delay; a bot making millions of requests pays the cost every time.



Anubis running a proof-of-work challenge in the browser before granting access.

Message authentication codes

Message Authentication Code (MAC)

A MAC is a cryptographic function that takes a key K and a message M and produces a tag T that authenticates the message. Only someone with the same key can verify the tag.

- A MAC provides **integrity** and **authenticity** for messages.
- Unlike hash functions, MACs require a secret key.
- MACs address the malleability problem we saw with encryption schemes. Without a MAC, attackers could modify ciphertexts.
- A secure MAC should be unforgeable, even after seeing tags for chosen messages.

A super basic practical example: AES-CTR + HMAC

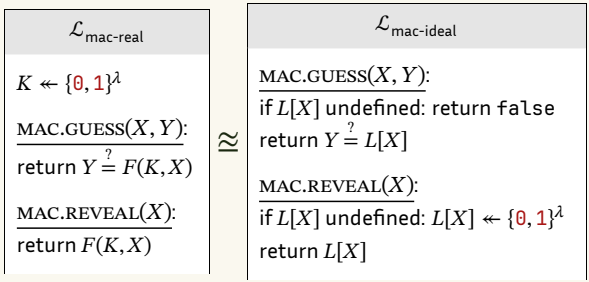
- Recall: AES-CTR alone is CPA-secure but **malleable**, so it's not CCA-secure.
- Fix with **Encrypt-then-MAC** using HMAC-SHA256:
  - Pick two independent keys: K_e for AES-CTR, K_m for HMAC.
  - Encrypt with AES-CTR, then HMAC the *ciphertext*.
  - On decryption, verify the tag **before** decrypting.
- Now any tampered ciphertext fails tag verification and is rejected.

$\underline{\text{ENC}((K_e, K_m), M)}$:
$C \coloneqq \texttt{AES-CTR.Enc}(K_e, M)$
$T \coloneqq \texttt{HMAC-SHA256}(K_m, C)$
return $C \| T$

$\underline{\text{DEC}((K_e, K_m), C\|T)}$:
if $\texttt{HMAC-SHA256}(K_m, C) \neq T$:
    return err
return $\texttt{AES-CTR.Dec}(K_e, C)$

PRFs as MACs

- A pseudorandom function (PRF) can be used directly as a MAC!
- The MAC key is the PRF key K.
- To authenticate a message X:
  - Compute the tag $T = F(K, X)$.
  - Send both X and T to the recipient.



From CPA to CCA security

The Encrypt-then-MAC approach

- CPA security isn't enough in the real world.
- We need protection against chosen-ciphertext attacks.
- Solution: combine encryption with a MAC!
- **Encrypt-then-MAC**:
  - Encrypt the message normally.
  - Compute a MAC tag of the *ciphertext*.
  - Send both ciphertext and tag.
  - Receiver verifies the tag before decrypting.
- This prevents adversaries from creating valid modified ciphertexts.

Encrypt-then-MAC: formal construction

Encrypt-then-MAC Construction

Let Σ be an SKE scheme and F be a PRF with output length λ whose domain includes $\Sigma.\mathcal{C}$. Define a new encryption scheme:

$$\mathcal{K} = \Sigma.\mathcal{K} \times \{0,1\}^\lambda$$
$$\mathcal{M} = \Sigma.\mathcal{M}$$
$$\mathcal{C}(\ell) = \Sigma.\mathcal{C}(\ell) \times \{0,1\}^\lambda$$

$\underline{\textsc{Enc}((K_e, K_m), M)}$:
$C \coloneqq \Sigma.\mathsf{Enc}(K_e, M)$
$T \coloneqq F(K_m, C)$
return $C \| T$

$\underline{\textsc{Dec}((K_e, K_m), C\|T)}$:
if $F(K_m, C) \neq T$: return $\mathtt{err}$
return $\Sigma.\mathsf{Dec}(K_e, C)$

CCA security of Encrypt-then-MAC

CCA Security Claim

If Σ is a CPA-secure encryption scheme and F is a secure PRF, then the Encrypt-then-MAC construction is CCA-secure.

- Key insight: **the MAC prevents tampering**.
- Without the MAC key, adversary can't create valid tags.
- **Decryption oracle only returns plaintexts for ciphertexts with valid tags.**

$\mathcal{L}_{\text{cca-real}}$

$K_e \leftarrow \Sigma.\mathcal{K}$
$K_m \leftarrow \{0,1\}^\lambda$

$\underline{\text{CCA.ENC}(M)}$:
$C := \Sigma.\text{Enc}(K_e, M)$
$T := F(K_m, C)$
return $C \| T$

$\underline{\text{CCA.DEC}(C \| T)}$:
if $F(K_m, C) \neq T$: return err
return $\Sigma.\text{Dec}(K_e, C)$

$\approx$

$\mathcal{L}_{\text{cca-rand}}$

$K_e \leftarrow \Sigma.\mathcal{K}$
$K_m \leftarrow \{0,1\}^\lambda$

$\underline{\text{CCA.ENC}(M)}$:
$C \leftarrow \Sigma.\mathcal{C}(|M|)$
$T \leftarrow \{0,1\}^\lambda$
$\mathcal{D}[C \| T] := M$
return $C \| T$

$\underline{\text{CCA.DEC}(C \| T)}$:
if $\mathcal{D}[C \| T]$ defined: return $\mathcal{D}[C \| T]$
if $F(K_m, C) \neq T$: return err
return $\Sigma.\text{Dec}(K_e, C)$

Combining MACs and encryption

- Not every way of combining a MAC and CPA-secure encryption achieves CCA security.
- There are three common approaches to combining them:
  - **Encrypt-then-MAC**: Encrypt message, then MAC the ciphertext.
  - **Encrypt-and-MAC**: Encrypt message and MAC the plaintext separately.
  - **MAC-then-encrypt**: MAC the plaintext, then encrypt both message and tag.
- Only one approach guarantees CCA security when using any CPA-secure encryption scheme.

Encrypt-then-MAC

CCA-secure

- **MAC verifies ciphertext integrity** before decryption.
- Prevents attackers from submitting modified ciphertexts.
- **Always CCA-secure** if encryption is CPA-secure and MAC is secure.

$\underline{\text{Enc}((K_e, K_m), M)}$:
$C \coloneqq \Sigma.\text{Enc}(K_e, M)$
$T \coloneqq F(K_m, C)$
return $C \| T$

$\underline{\text{Dec}((K_e, K_m), C \| T)}$:
if $F(K_m, C) \neq T$: return err
return $\Sigma.\text{Dec}(K_e, C)$

Encrypt-and-MAC

Not even CPA-secure!

- **MAC is computed on the plaintext**.
- Same plaintext always produces same tag, leaking equality information.
- **Not even CPA-secure**, let alone CCA-secure.

$\underline{\textsc{Enc}((K_e, K_m), M)}$:
$C \coloneqq \Sigma.\mathsf{Enc}(K_e, M)$
$T \coloneqq F(K_m, M)$
return $C \| T$

$\underline{\textsc{Dec}((K_e, K_m), C \| T)}$:
$M \coloneqq \Sigma.\mathsf{Dec}(K_e, C)$
if $F(K_m, M) \neq T$: return $\mathtt{err}$
return M

MAC-then-encrypt

It's complicated

- **Tag is hidden inside the ciphertext**.
- Whether this is CCA-secure depends on the specific encryption scheme.
- **Not generally CCA-secure** for all CPA-secure encryption schemes.

$\underline{\text{Enc}((K_e, K_m), M)}$:
$T := F(K_m, M)$
$C := \Sigma.\text{Enc}(K_e, M \| T)$
return C

$\underline{\text{Dec}((K_e, K_m), C)}$:
$M \| T := \Sigma.\text{Dec}(K_e, C)$
if $F(K_m, M) \neq T$: return err
return M

Encrypt + MAC security comparison

Construction	CPA-secure?	CCA-secure?
Encrypt-then-MAC	Yes	Yes
Encrypt-and-MAC	No	No
MAC-then-encrypt	Yes	Maybe

- **Encrypt-then-MAC** is the safest option.
- **Encrypt-and-MAC** should never be used.
- **MAC-then-encrypt** requires case-by-case analysis.

Section 2

Chosen-Ciphertext Attacks

Subsection 2.2

Authenticated Encryption

Authenticated Encryption: beyond CCA security

- CCA security is stronger than CPA security, but still not the gold standard.
- CCA security says: adversary-generated ciphertexts won't reveal useful information.
- But CCA security doesn't require that they decrypt to `err`.
- For many applications, we want a stronger guarantee:
  - Only key-holders can create valid ciphertexts.
  - All other ciphertexts should be rejected as invalid.
- This property is called **Authenticated Encryption (AE)**.
- Authenticated encryption provides both confidentiality and authenticity.

Authenticated Encryption: formal definition

Authenticated Encryption

An SKE scheme Σ is a secure authenticated encryption (AE) scheme if the following two libraries are indistinguishable:

$\mathcal{L}^{\Sigma}_{\text{ae-real}}$

$K \twoheadleftarrow \Sigma.\mathcal{K}$

$\underline{\text{AE.ENC}(M)}$:
return $\Sigma.\text{Enc}(K, M)$

$\underline{\text{AE.DEC}(C)}$:
return $\Sigma.\text{Dec}(K, C)$

$\approx$

$\mathcal{L}^{\Sigma}_{\text{ae-rand}}$

$\underline{\text{AE.ENC}(M)}$:
$C \twoheadleftarrow \Sigma.\mathcal{C}(|M|)$
$\mathcal{D}[C] \coloneqq M$
return C

$\underline{\text{AE.DEC}(C)}$:
if $\mathcal{D}[C]$ defined: return $\mathcal{D}[C]$
else: return err

AE vs. CCA security

- Key difference: **how we handle adversary-created ciphertexts**.
- In $\mathcal{L}^{\Sigma}_{\text{ae-rand}}$, any ciphertext not created by the library always decrypts to `err`.
- In $\mathcal{L}^{\Sigma}_{\text{cca-rand}}$, such ciphertexts could decrypt to anything (not necessarily `err`).
- So AE requires:
  - Adversary cannot tell real from random ciphertexts (as in CPA).
  - Adversary cannot create new valid ciphertexts (authentication).

- AE is **strictly stronger** than CCA security.
- Every AE scheme is CCA-secure, but not every CCA-secure scheme is an AE scheme.
- Making the distinction explicit helps us design better protocols.
- AE is what you should aim for in practice.

How Encrypt-then-MAC achieves AE

- Remember our Encrypt-then-MAC construction:
  - Encrypt the plaintext: $C \coloneqq \Sigma.\mathsf{Enc}(K_e, M)$
  - MAC the ciphertext: $T \coloneqq F(K_m, C)$
  - Send both: $C \| T$.
- It achieves AE security because:
  - Without K_m, adversary can't forge valid tags.
  - Any ciphertext not created by the system will fail MAC verification.
  - MAC verification failures lead to `err`.
- The proof is nearly identical to the CCA security proof.

AE Security of Encrypt-then-MAC

Encrypt-then-MAC is a secure AE if the underlying Σ is a CPA-secure SKE and F is a secure PRF.

AE implies CCA

If an encryption scheme Σ is a secure AE, then it is also CCA-secure.

Authenticated Encryption: in practice

- Modern cryptographic protocols almost always use authenticated encryption.
- Common AE implementations:
  - **AES-GCM** (Galois/Counter Mode): most widely used, combines CTR mode with a MAC.
  - **ChaCha20-Poly1305**: popular alternative to AES-GCM, especially on devices without AES hardware.
  - **AES-CBC + HMAC-SHA256**: older approach, uses Encrypt-then-MAC with AES in CBC mode.
- Important implementation rule: **verify before decrypt**!
  - Always check the MAC before decrypting.
  - Prevents timing side-channels based on decryption behavior.
  - Helps protect against padding oracle attacks and similar vulnerabilities.

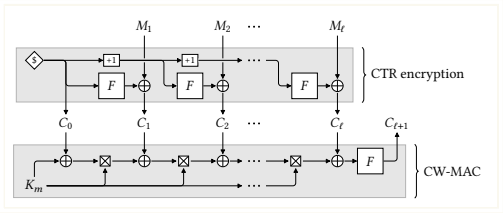
AES-GCM: Galois/Counter Mode

- AES-GCM (Galois/Counter Mode) is the most widely used AEAD scheme.
- Combines AES in CTR mode (for encryption) with GMAC (for authentication).
- Extremely efficient:
  - Single pass over the data.
  - **Parallelizable**.
  - Hardware acceleration widely available.
- Used in: TLS 1.2/1.3, IPsec, SSH, and many other protocols.

$\underline{\text{AES-GCM.Enc}(K, N, A, M)}$:
$H \coloneqq \mathtt{AES}(K, 0^{128})$
for $i = 1$ to $\lceil |M|/128 \rceil$:
  $C_i \coloneqq M_i \oplus \mathtt{AES}(K, N \| i)$
$T \coloneqq \mathtt{GHASH}_H(A, C) \oplus$
  $\mathtt{AES}(K, N \| 0)$
return $C \| T$

AES-GCM: Galois/Counter Mode

- **Inputs**:
  - Key K (128, 192, or 256 bits)
  - Nonce N (usually 96 bits)
  - Associated data A (optional)
  - Plaintext M
- **Encryption process**:
  - AES-CTR for confidentiality.
  - GHASH (Galois field[a] multiplication) for authentication.
- The tag T protects both ciphertext and associated data.



Source: The Joy of Cryptography

[a] "Galois field" is another name for finite fields. We use those for Diffie-Hellman too, remember?

What is GHASH?

- **GHASH** is a universal hash function based on Galois field arithmetic.
- Key properties:
  - Operates in the finite field $GF(2^{128})$.
  - Uses polynomial multiplication modulo an irreducible polynomial.
  - Extremely fast with hardware support.
- How it works:
  - Treats input blocks as coefficients of a polynomial.
  - Evaluates the polynomial at a secret point $H = \text{AES}(K, 0^{128})$.
  - Result: $\text{GHASH}_H(X) = X_1 \cdot H^n + X_2 \cdot H^{n-1} + \ldots + X_n \cdot H$.

Galois Field $GF(2^{128})$

- Elements: 128-bit strings
- Addition: XOR
- Multiplication: polynomial multiplication mod $x^{128} + x^7 + x^2 + x + 1$
- Every non-zero element has an inverse

Why GHASH instead of HMAC?

- **Why not use HMAC-SHA256 or similar?**
  - HMAC would work, but it's much slower.
  - HMAC requires two nested hash computations.
  - SHA-256 is inherently sequential.
- **GHASH advantages:**
  - Single multiplication per block (after setup)
  - **Fully parallelizable** across blocks
  - Hardware acceleration: Intel `PCLMULQDQ` instruction
  - Can process at line speed (10+ Gbps)
- **Mathematical elegance:**
  - Galois field operations have nice algebraic properties.
  - Security proofs are cleaner.

Performance Comparison

For a 1KB message:

- **HMAC-SHA256**: ~3,000 cycles
- **GHASH (software)**: ~1,500 cycles
- **GHASH (hardware)**: ~200 cycles

GHASH can be 15× faster with hardware support!

If GHASH is so great, why not use it everywhere?

- **GHASH limitations:**
  - **Requires a secret key** - GHASH is a keyed function.
  - **Not collision-resistant** without the key - anyone can find collisions trivially.
  - **Linear structure** - $\text{GHASH}_H(A \oplus B) = \text{GHASH}_H(A) \oplus \text{GHASH}_H(B)$.
  - **Forgeable if key is known** - completely breaks down with key exposure.
- **When we need SHA-2/SHA-3:**
  - **Public verification**: digital signatures, certificates, Git commits
  - **Proof of work**: Bitcoin mining, anti-spam
  - **Key derivation**: generating keys from passwords (PBKDF2)
- **Different tools for different jobs:**
  - GHASH: Fast MAC when you have a secret key
  - SHA-2/3: Collision-resistant hash when you need public verifiability

AES-GCM: Galois/Counter Mode

- **Security strengths**:
  - Provides confidentiality, integrity, and authenticity.
  - Formally proven secure (assuming AES is secure).
  - Fast and widely trusted.
- **Critical implementation requirements**:
  - **Never reuse a nonce** with the same key!
  - A repeated nonce can lead to complete loss of confidentiality and authentication.
  - 96-bit nonces are recommended (other sizes are less efficient).
  - Use the full 128-bit authentication tag (don't truncate it).

Key commitment in authenticated encryption

- **Key commitment**: a ciphertext should only decrypt to a valid plaintext under the key used to generate it.
- Most AEAD schemes (including AES-GCM) don't guarantee this property!
- Attack scenario:
  1. The attacker creates a special ciphertext C.
  2. When decrypted with key K_1: harmless message.
  3. When decrypted with key K_2: malicious content!
  4. Enables plausible deniability, content smuggling, etc.
- Practical impact:
  - An attacker can create ciphertexts that decrypt differently under different keys.
  - Enables attacks in multi-recipient contexts.
  - Affects real applications (e.g., messaging, encrypted files).

ChaCha20-Poly1305

- ChaCha20-Poly1305 is a modern AEAD construction:
  - ChaCha20 stream cipher for encryption.
  - Poly1305 MAC for authentication.
- Designed by Daniel J. Bernstein.[a]
- Key characteristics:
  - No table lookups (better resistance to timing attacks).
  - Excellent performance on devices without AES hardware.
- Widely used in TLS 1.3, Signal, WireGuard...

ChaCha20-Poly1305.Enc(K, N, A, M):

$\text{key}_p \coloneqq \texttt{ChaCha20}(K, N, 0)_{0..31}$

$C \coloneqq M \oplus \texttt{ChaCha20}(K, N, 1)$

$\text{data} \coloneqq \text{pad}(A) \| \text{pad}(C) \| \text{len}(A) \| \text{len}(C)$

$T \coloneqq \texttt{Poly1305}_{\text{key}_p}(\text{data})$

return $C \| T$

[a] Does the class want to hear about Bernstein vs. United States?

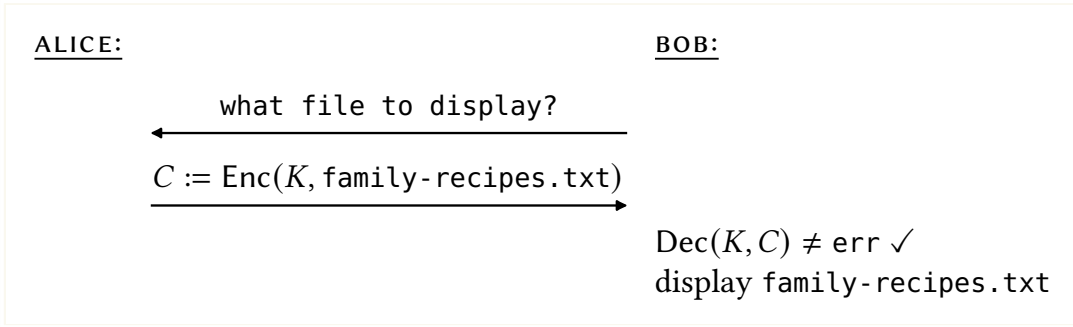
Comparing AEAD implementations

Property	AES-GCM	ChaCha20-Poly1305	AES-CBC+HMAC
Performance (HW accel.)	Good	Good	Okay
Performance (no accel.)	Okay	Good	Okay
Security level	128-256 bits	256 bits	128-256 bits
Side-channel resistance	Moderate	Excellent	Moderate
Parallelizable	Yes	Partially	No
Nonce sensitivity	Very high	High	Moderate
Overhead	Low (1 pass)	Low (1 pass)	Higher (2 pass)
Implem. complexity	Moderate	Low	Low

Also, check out: `https://soatok.blog/2020/07/12/comparison-of-symmetric-encryption-methods/`

Replay attacks

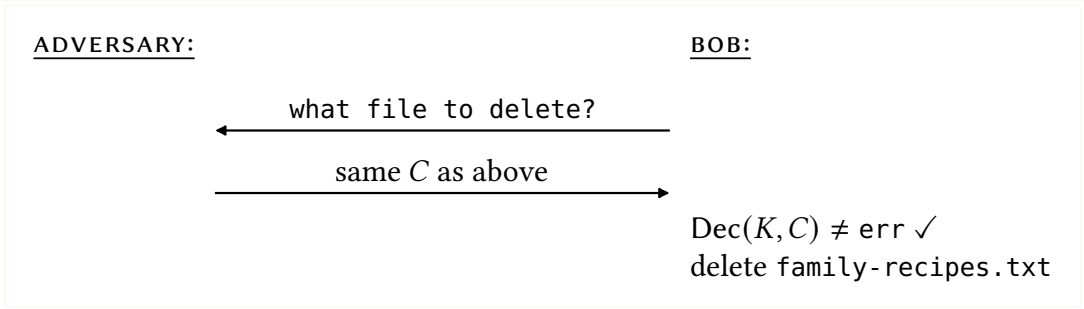
Step 1: Alice sends original message



Source: The Joy of Cryptography

Replay attacks

Step 2: Attacker replays message in a different context



Source: The Joy of Cryptography

Replay attacks

Authenticated encryption didn’t save us

- Even with authenticated encryption, context matters!
- Scenario: Alice sends Bob encrypted commands.
  - Each ciphertext contains Alice’s genuine intent.
  - Bob trusts any ciphertext that decrypts successfully.
- Vulnerability: An adversary can replay legitimate ciphertexts.
  - Alice once sent “Delete temporary files”.
  - Adversary replays it when Alice meant to say “Display files”.

Associated data

- Solution 1: Include context in the plaintext.
  - "`ACTION: DISPLAY`" before the actual message.
  - Inefficient - increases message size.
  - Both parties already know the context.
- Better solution: Associated Data (AD).
  - Context that sender and receiver already know.
  - Used during encryption and decryption.
  - Doesn't increase ciphertext size.
- How it works:
  - $\mathsf{Enc}(K, A, M) \to C$ where A is associated data.
  - $\mathsf{Dec}(K, A, C) \to M$ or `err`.

AEAD: formal definition

AEAD

An encryption scheme Σ with associated data is a secure authenticated encryption with associated data (AEAD) scheme if the following two libraries are indistinguishable:

$\mathcal{L}^{\Sigma}_{\text{aead-real}}$

$K \twoheadleftarrow \Sigma.\mathcal{K}$

$\underline{\text{AEAD.ENC}(A, M)}$:
return $\Sigma.\text{Enc}(K, A, M)$

$\underline{\text{AEAD.DEC}(A, C)}$:
return $\Sigma.\text{Dec}(K, A, C)$

$\approx$

$\mathcal{L}^{\Sigma}_{\text{aead-rand}}$

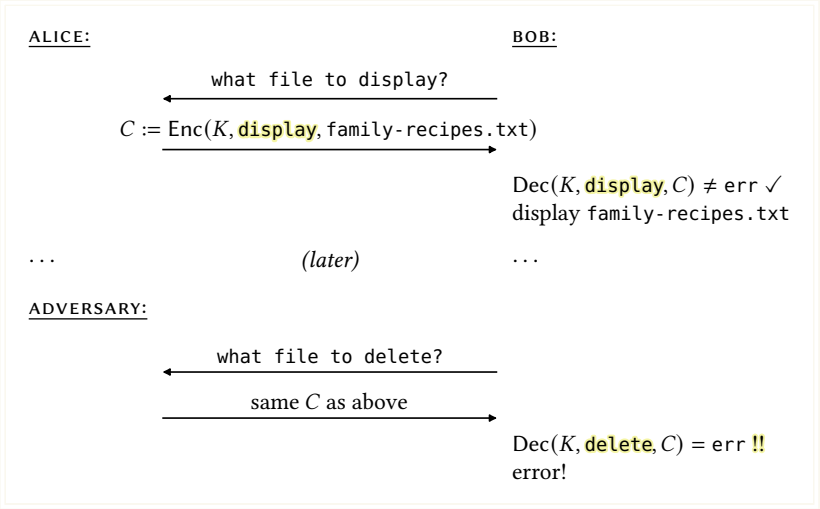
$\underline{\text{AEAD.ENC}(A, M)}$:
$C \twoheadleftarrow \Sigma.\mathcal{C}(|M|)$
$\mathcal{D}[A, C] \coloneqq M$
return C

$\underline{\text{AEAD.DEC}(A, C)}$:
if $\mathcal{D}[A, C]$ defined:
  return $\mathcal{D}[A, C]$
else: return err

Usefulness of AEADs

- An AEAD (Authenticated Encryption with Associated Data) scheme guarantees:
  - Ciphertexts reveal nothing about plaintexts (confidentiality).
  - Only ciphertexts created by the legitimate sender will decrypt without error (authenticity).
  - Decryption only succeeds when the correct associated data is used (context binding).
- What to use as associated data?
  - Protocol information: “`DISPLAY`” vs. “`DELETE`”.
  - Session identifiers or timestamps.
  - Previous messages in the conversation.
  - Any contextual information both parties already know.
- Use as much associated data as relevant - it’s cryptographically “free”!

Solution: use associated data to provide context



Source: The Joy of Cryptography

The Fujisaki-Okamoto Transform

From CPA to CCA in public-key encryption (optional slide)

- What about public-key encryption?
- Many public-key schemes (RSA, ElGamal) are only CPA-secure.
- The **Fujisaki-Okamoto (FO) transform** converts CPA-secure PKE to CCA-secure PKE.
- Basic idea:
  - Use randomness derived from the message itself.
  - Add redundancy that can be verified during decryption.
  - Invalid ciphertexts will fail verification.

FO Transform (simplified)

Given a CPA-secure PKE scheme Π:

- $\mathrm{Enc}_{FO}(pk, M)$:
  1. $R := H(M)$
  2. Return $C := \Pi.\mathrm{Enc}(pk, M; R)$
- Decryption verifies consistency by re-encrypting M with $H(M)$

現代與後量子密碼學導論

Introduction to Modern and Post-quantum Cryptography

Fall 2026

Part 1: Provable Security

1.5: Chosen-Plaintext & Chosen-Ciphertext Attacks

Zi-Yuan Liu

CC BY NC SA

The original slides were prepared by Nadim Kobeissi.